

ΒΑΣΙΛΕΙΟΣ ΚΟΚΚΙΝΟΓΕΝΗΣ IT2021042

ΕΡΓΑΣΙΑ ΣΤΟ IR & NLP

Δημιουργία μιας μηχανής αναζήτησης πάνω σε wikipedia movie plots

Η δημιουργία μιας μηχανής αναζήτησης χωρίζεται σε 5 κομμάτια

1. Loading/Preprocessing
2. Embeddings
3. Indexing
4. Retrieval And Ranking
5. Golden Answers And Evaluation

1. Αρχικά, για το loading και preprocessing του csv αρχείου με τις ταινίες έγιναν τα εξής:

- **Ανάγνωση Αρχείων:**
Φορτώνω ολόκληρο το σύνολο δεδομένων χρησιμοποιώντας pandas.
- **Δημιουργία ID για κάθε έγγραφο:**
Αντιστοιχίζω ένα μοναδικό doc_id σε κάθε γραμμή όπως doc_0, doc_1, κ.λπ., για να μπορώ να αναγνωρίζω κάθε έγγραφο.
- **Καθαρισμός Δεδομένων:**
Αντικαθιστώ τιμές όπως 'unknown' με NaN χρησιμοποιώντας μια βοηθητική συνάρτηση και ελέγχω για τιμές που λείπουν σε κάθε στήλη.
- **Αφαίρεση Διπλότυπων:**
Αφαιρώ ακριβή διπλότυπα με βάση τον συνδυασμό Τίτλου και Έτους Κυκλοφορίας για να αποφύγω τα duplicates
- **Επεξεργασία Γραμμών:**
Για κάθε γραμμή, εξάγω σημαντικά πεδία (όπως τίτλος, πλοκή, είδος κ.λπ.) και δημιουργώ ένα νέο λεξικό. Δημιουργώ επίσης μια συνδυασμένη συμβολοσειρά αξιολόγησης από τον τίτλο, την πλοκή και το είδος για να χρησιμοποιηθεί αργότερα για evaluation.
- **Τελικό Αποτέλεσμα:** Το αποτέλεσμα είναι ένα δομημένο λεξικό (docs_metadata) που αντιστοιχίζει κάθε doc_id στα καθαρισμένα και επεξεργασμένα μεταδεδομένα του.

2. Για την δημιουργία αναπαραστάσεων των κειμένων έγιναν τα εξής:

α) SBERT Embeddings

- **Επιλεγμένα Πεδία:**
Ορίζω μια λίστα βασικών πεδίων όπως τίτλος, πλοκή, είδος και αξιολόγηση που καταγράφουν σημαντικές πληροφορίες κειμένου για κάθε ταινία.
- **Φιλτράρισμα & Συλλογή Κειμένων:**
Για κάθε πεδίο, εξάγω μόνο τις μη κενές τιμές κειμένου από τα μεταδεδομένα και τις αντιστοιχίζω με τα αντίστοιχα doc_ids.
- **Δημιουργία Ενσωματώσεων:**
Χρησιμοποιώ το παρεχόμενο μοντέλο ενσωμάτωσης προτάσεων (all-MiniLM-L6-v2) για να κωδικοποιήσω όλα τα κείμενα σε αριθμητικά διανύσματα με batch size 128
- **Id Mappings:**
Οι ενσωματώσεις που προκύπτουν αντιστοιχίζονται στα doc_ids τους, διασφαλίζοντας ότι κάθε ενσωμάτωση συνδέεται με το σωστό έγγραφο.
- **Αποθήκευση σε Δίσκο:**
Αποθηκεύω όλες τις ενσωματώσεις (για όλα τα πεδία) σε ένα ένθετο λεξικό χρησιμοποιώντας το pickle, καθιστώντας εύκολη τη φόρτωσή τους αργότερα για να αποφύγουμε να τις ξαναδημιουργήσουμε.

β) BM25

- **Επιλεγμένα Πεδία:**
Ορίζω μια λίστα βασικών πεδίων όπως τίτλος, πλοκή, είδος και αξιολόγηση που καταγράφουν σημαντικές πληροφορίες κειμένου για κάθε ταινία.
- **Tokenization:**
Για κάθε έγγραφο και κάθε πεδίο, κάνω tokenization στο κείμενο χρησιμοποιώντας μια προσαρμοσμένη συνάρτηση tokenization, αφαιρώντας τα παρεχόμενα stopwords. Το αποτέλεσμα είναι ένα ανά πεδίο λεξικό λιστών token, μία λίστα ανά doc_id.

- **Corpus Indexing:**
Αποθηκεύω αυτά τα tokenized κείμενα σε ένα corpus index ανά πεδίο, το οποίο αργότερα θα επιτρέψει την αποτελεσματική κατάταξη των εγγράφων.
 - **Υπολογισμός IDF:**
Για κάθε πεδίο, υπολογίζω τις τιμές Inverse Document Frequency (IDF) για όλους τους μοναδικούς όρους, καταγράφοντας πόσο σπάνιο ή κοινό είναι κάθε token σε όλα τα έγγραφα.
 - **Αποθήκευση σε Δίσκο:**
Τόσο τα corpus indexes όσο και οι τιμές IDF αποθηκεύονται χρησιμοποιώντας pickle, καθιστώντας εύκολη τη φόρτωσή τους αργότερα για να αποφύγουμε να τα ξαναδημιουργήσουμε.
3. Για την δημιουργία index (SBERT only) χρησιμοποιήθηκε faiss index και έγιναν τα εξής:
- **Επεξεργασία Κάθε Πεδίου:**
Για κάθε πεδίο μεταδεδομένων (όπως τίτλος, γράφημα, κ.λπ.), στοιβάζω όλες τις ενσωματώσεις προτάσεων σε έναν πίνακα.
 - **Κανονικοποίηση για Αναζήτηση Ομοιότητας:**
Εφαρμόζω κανονικοποίηση L2 ώστε να μπορούμε να χρησιμοποιήσουμε το εσωτερικό γινόμενο (cosine similarity) ως μέτρο ομοιότητας.
 - **Δημιουργία και Συμπλήρωση Ευρετηρίου:**
Δημιουργώ ένα ευρετήριο FlatIP (Inner Product) και προσθέτω τα embeddings.
 - **Αποθήκευση Ευρετηρίου:**
Το ευρετήριο αποθηκεύεται σε ένα αρχείο καθιστώντας εύκολη τη φόρτωσή του αργότερα για να αποφύγουμε να το ξαναδημιουργήσουμε.

4. Για τον μηχανισμό ανάκτησης και βαθμονόμησης των αποτελεσμάτων έγιναν τα εξής:

α) SBERT Embeddings

- **Query Encoding:**
Μετατρέπω το input query (`q_text`) σε ενσωμάτωση χρησιμοποιώντας το ίδιο μοντέλο που χρησιμοποιήθηκε για τα έγγραφα.
- **Document Embedding Retrieval:**
Ανακτώ όλες τις προ-υπολογισμένες ενσωματώσεις εγγράφων και τα αντίστοιχα `doc_ids`.
- **Αναζήτηση βάσει FAISS (Γρήγορη Επιλογή):**
Ομαλοποιώ την ενσωμάτωση ερωτήματος και την αναζητώ σε σχέση με τον δείκτη FAISS. Το FAISS επιστρέφει από τα top 10 documents τις βαθμολογίες ομοιότητάς τους και τους δείκτες (indices). Αντιστοιχίζω αυτούς τους δείκτες πίσω στα `doc_ids` τους και επιστρέφω τα 10 πιο παρόμοια έγγραφα.
- **Εφεδρική Αναζήτηση (Αργή Επιλογή):**
Εάν δεν χρησιμοποιείται το FAISS υπολογίζω την ομοιότητα συνημίτονου μεταξύ της ενσωμάτωσης ερωτήματος και των ενσωματώσεων εγγράφων. Στη συνέχεια, ταξινομώ και επιστρέφω τα 10 πιο παρόμοια έγγραφα.

β) BM25

- **Tokenized the Query:**
Χωρίζω το κείμενο εισόδου `q_text` σε tokens (λέξεις), αφαιρώντας τα stopwords για καλύτερη αντιστοίχιση.
- **Βαθμολογία Κάθε Εγγράφου:**
Για κάθε έγγραφο στο σώμα κειμένων υπολογίζω μια βαθμολογία συνάφειας BM25 συγκρίνοντας τη συχνότητα εμφάνισης κάθε όρου ερωτήματος. Η βαθμολογία χρησιμοποιεί τη συχνότητα όρου (TF), την αντίστροφη συχνότητα εγγράφου (IDF) και την κανονικοποίηση μήκους εγγράφου.
- **Κατάταξη Εγγράφων:**
Ταξινομώ όλα τα έγγραφα με βάση τη βαθμολογία τους σε φθίνουσα σειρά και επιστρέφω τα 10 πιο σχετικά.

5. Για την αξιολόγηση της συνάφειας των πρώτων 10 αποτελεσμάτων που επιστρέφονται σε κάθε ερώτηση έγιναν τα εξής:

α) Αρχικά, για να μπορέσουμε να αξιολογήσουμε την συνάφεια των αποτελεσμάτων πρέπει πρώτα να έχουμε απαντήσεις ειδικών στις οποίες μπορούμε να βασιστούμε για να κρίνουμε εάν κάποιο έγγραφο είναι σχετικό ή όχι. Τις απαντήσεις αυτές τις παίρνουμε μέσω ενός *llm as a judge* κάνοντας τα εξής:

- **Δημιουργία prompt:**
Δίνουμε το query και τα top 10 documents στο prompt ζητώντας από το LLM να επισημάνει κάθε έγγραφο ως "ναι" (σχετικό) ή "όχι".
- **Αποστολή στο Hugging Face API:**
Το prompt αποστέλλεται στο Mistral-7B-Instruct μέσω του Hugging Face API χρησιμοποιώντας ένα api key.
- **Parse LLM Response:**
Η απάντηση του μοντέλου αναλύεται γραμμή προς γραμμή σε μια λίστα *relevance judgements*
- **Επισύναψη judgement σε έγγραφα:**
Κάθε έγγραφο επισημαίνεται με την προβλεπόμενη ετικέτα συνάφειας (ναι ή όχι).
- **Αποθήκευση των Golden Answers:**
Μόνο τα έγγραφα που έχουν επισημανθεί ως "ναι" αποθηκεύονται στο αρχείο *golden_answers* για χρήση στην αξιολόγηση της συνάφειας.

β) Εφόσον έχουμε τα golden answers σε ένα αρχείο και τα φορτώσουμε, μπορούμε να κάνουμε αξιολόγηση συνάφειας ως εξής:

- **Σύγκριση με Golden Answers:**
Για κάθε ερώτημα, συγκρίνουμε τα κορυφαία έγγραφα που ανακτήθηκαν με τα golden answers. Υπολογίζουμε τα σχετικά έγγραφα που ανακτήθηκαν σωστά (TP) και τα άσχετα έγγραφα που ανακτήθηκαν εσφαλμένα (FP) και υπολογίζουμε την ακρίβεια ως $TP / (TP + FP)$
- **Συνολικές Βαθμολογίες:**
Υπολογίζουμε τη μέση ακρίβεια σε όλα τα ερωτήματα για να λάβουμε μια συνολική βαθμολογία απόδοσης.

Για τα 30 queries που έδωσα στην μηχανή αναζήτησης είχα τα εξής αποτελέσματα:

```
Model: all-MiniLM-L6-v2
Mean Precision: 0.4667
Total time for all queries for model all-MiniLM-L6-v2 : 8.674 seconds
Total time for all queries for model all-MiniLM-L6-v2 with index : 2.073 seconds

Model: bm25
Mean Precision: 0.4100
Total time for all queries for model bm25 : 9.421 seconds
```

Παρατηρούμε ότι Το MiniLM με ευρετηρίαση FAISS είναι η καλύτερη επιλογή εφόσον έχει υψηλότερη ακρίβεια και πολύ ταχύτερη ανάκτηση. Ωστόσο το BM25, αν και απλούστερο, παραμένει ανταγωνιστικό τόσο σε ακρίβεια όσο και σε ταχύτητα (αν το συγκρίνουμε με το MiniLM χωρίς FAISS).

Περαιτέρω βελτιώσεις:

Θα μπορούσαμε να εξερευνήσουμε και να συγκρίνουμε εναλλακτικά μοντέλα ενσωματώσεων όπως το GloVe, το Doc2Vec κλπ, για να αξιολογήσουμε ποιες αναπαραστάσεις έχουν καλύτερη απόδοση στην ανάκτηση σχετικών κειμένων.

Έξτρα:

Ανέπτυξα μια διεπαφή μηχανής αναζήτησης που στέλνει αιτήματα στον server χρησιμοποιώντας HTML, όπου οι χρήστες μπορούν να:

- Εισαγάγουν ένα query
- Επιλέξουν ένα πεδίο μεταδεδομένων για αναζήτηση (π.χ., τίτλος, πλοκή, ηθοποιοί)
- Επιλέξουν μεταξύ αναζήτησης βάσει λέξεων-κλειδιών (BM25) και σημασιολογικής αναζήτησης (χρησιμοποιώντας ενσωματώσεις προτάσεων)
- Λάβουν τα top 10 documents για το συγκεκριμένο query και αν είναι relevant ή όχι

Επίσης ανέπτυξα έναν Flask server που δέχεται αιτήματα από τους χρήστες και προσφέρει 2 endpoints

1) /search

- Επεξεργάζεται τα ερωτήματα των χρηστών σε πραγματικό χρόνο. Με βάση την επιλεγμένη μέθοδο αναζήτησης και το πεδίο μεταδεδομένων εκτελεί είτε κατάταξη BM25 είτε σημασιολογική ομοιότητα με FAISS
- Επιστρέφει τα αποτελέσματα με την υψηλότερη κατάταξη μαζί με τα πλήρη μεταδεδομένα εγγράφων (τίτλος, προέλευση, σκηνοθέτης, είδος, πλοκή κ.λπ.)

2) /judge

- Αποστέλλει το query και τα top 10 αποτελέσματα σε ένα γλωσσικό μοντέλο (LLM) μέσω API
- Λαμβάνει κρίσεις όπως "Εγγραφο 1: ναι" ή "Εγγραφο 2: όχι" για να υποδείξει εάν κάθε έγγραφο είναι σχετικό
- Επιστρέφει τα documents μαζί με το αν είναι relevant ή όχι

Tenies Online

Search movies:

movies with lots of fighting

Plot



Semantic Based



Search

Judge

(4.821s)

Sekigahara (2017)

The film is a Japanese period drama about the Battle of Sekigahara, a six-hour battle in 1600 that brought an end to the Warring States era in Japanese history.[2]

Origin: Japanese

Director: Masato Harada

Cast: Junichi Okada, Kōji Yakusho, Kasumi Arimura, Takehiro Hira, Masahiro Higashide, Yukiya Kitamura

Genre: historical, fantasy, drama, jidaigeki, action

Bless Me, Ultima (2013)

A drama set in New Mexico during World War II, centered on the relationship between a young man and an elderly medicine woman who helps him contend with the battle between good and evil that rages in his village.

Origin: American

Director: Carl Franklin

Cast: Luke Ganalon, Míriam Colón, Benito Martinez, Dolores Heredia

Genre: drama

The Bloody Fight (1972)

The evil Japanese are challenging kung fu masters to death duels and they are winning until one school learns to adapt and overcome by combining styles. Some drunken boxing helps along the